FUNDAMENTOS DE PROGRAMACIÓN. Curso 2024/25

SEGUNDO EXAMEN PARCIAL. 28 de mayo de 2025. Soluciones

Ejercicio 1: Tipo Producto (0,5 ptos)

```
public record Producto(String nombre, Double precio) {
	public Producto {
		Checkers.checkNoNull(nombre, precio);
		Checkers.check("El precio no puede ser negativo", precio > 0);
	}
}
```

Ejercicio 2: tipo CasetaFeria (1,5 ptos)

```
public class CasetaFeria implements Comparable<CasetaFeria> {
	private String nombre;
	private TipoCaseta tipo;
	private Integer modulo;
	private Integer aforo;
	private Set<Producto> menu;
	private String musica;
	private Boolean accesoAdaptado;
	private Double valoracion;

	public static final Double PRECIO_BARATO = 10.0;

	public CasetaFeria(String nombre, TipoCaseta tipo, Integer modulo,
			Integer aforo, Set<Producto> menu, String musica,
			Boolean accesoAdaptado, Double valoracion) {
		this.nombre = nombre;
		this.tipo = tipo;
		this.modulo = modulo;
		this.aforo = aforo;
		Checkers.check("El menú no puede estar vacío", menu.size()>0);
		this.menu = new HashSet<>(menu);
		setMusica(musica);
		this.accesoAdaptado = accesoAdaptado;
		setValoracion(valoracion);
	}

	public Integer getAforo() {
		return aforo;
	}

	public String getMusica() {
		return musica;
	}

	public void setMusica(String musica) {
		this.musica = musica;
	}

	public Double getValoracion() {
		return valoracion;
	}

	public void setValoracion(Double valoracion) {
		Checkers.check("La valoración debe estar entre 0 y 100",
			valoracion >= 0 && valoracion <= 100);
		this.valoracion = valoracion;
	}
```

```java
public String getNombre() {
	return nombre;
}

public TipoCaseta getTipo() {
	return tipo;
}

public Integer getModulo() {
	return modulo;
}

public Set<Producto> getMenu() {
	return new HashSet<>(menu);
}

public Boolean getAccesoAdaptado() {
	return accesoAdaptado;
}

public Double getPrecioMedio() {
	return menu.stream()
			.mapToDouble(Producto::precio)
			.average()
			.getAsDouble();
}

//También con bucles
public Double getPrecioMedio() {
	Double suma = 0.;
	for (Producto p: menu) {
		suma += p.precio();
	}
	Double res = null;
	if (menu.size()>0) {
		res = suma / menu.size();
	} else {
		throw new NoSuchElementException();
	}
	return res;
}

public Boolean getCasetaBarata() {
	return menu.stream()
			.allMatch(p -> p.precio() < PRECIO_BARATO);
}

//También con bucles
public Boolean getCasetaBarata() {
	Boolean res = true;
	for (Producto p: getMenu()) {
		if(!(p.precio() < PRECIO_BARATO)) {
			res = false;
			break;
		}
	}
	return res;
}
```

```
	public String toString() {
		return "CasetaFeria [getMusica()=" + getMusica()
			+ ", getValoracion()=" + getValoracion()
			+ ", getNombre()=" + getNombre()
			+ ", getTipo()=" + getTipo()
			+ ", getAforo()=" + getAforo()
			+ ", getModulo()=" + getModulo()
			+ ", Num productos menu=" + menu.size()
			+ ", getAccesoAdaptado()=" + getAccesoAdaptado() + "]";
	}

	public int hashCode() {
		return Objects.hash(modulo, nombre);
	}

	public boolean equals(Object obj) {
		if (this == obj)
			return true;
		if (obj == null)
			return false;
		if (getClass() != obj.getClass())
			return false;
		CasetaFeria other = (CasetaFeria) obj;
		return Objects.equals(modulo, other.modulo)
			&& Objects.equals(nombre, other.nombre);
	}

	public int compareTo(CasetaFeria cf) {
		int res = getModulo().compareTo(cf.getModulo());
		if (res == 0) {
			res= getNombre().compareTo(cf.getNombre());
		}
		return res;
	}
}
```

Ejercicio 3: Factoría (1 pto)

```
	private static final String SEPARADOR_PRINCIPAL = ";";
	private static final String SEPARADOR_MENU = ",";
	private static final String SEPARADOR_PRODUCTO = "-";

	//EL 88 DE COSTILLARES;publica;113;150;rebujito - 7.5 ,
	  tortilla patatas - 10 , jamon - 20;flamenco;si;87
	public static CasetaFeria parseaCaseta(String casetaStr) {
		Checkers.checkNoNull(casetaStr);
		String [] trozos = casetaStr.split(SEPARADOR_PRINCIPAL);
		Checkers.check("Formato no válido <"+casetaStr+">",
				trozos.length == 8);
		String nombre = trozos[0].trim();
		TipoCaseta tipo = parseaTipo(trozos[1].trim());
		Integer modulo = Integer.parseInt(trozos[2].trim());
		Integer aforo = Integer.parseInt(trozos[3].trim());
		Set<Producto> menu = parseaMenu (trozos[4].trim());
		String musica = trozos[5].trim();
		Boolean adaptado = parseaAdaptado(trozos[6].trim());
		Double valoracion = Double.parseDouble(trozos[7].trim());
		return new CasetaFeria(nombre, tipo, modulo, aforo,menu,
			musica, adaptado, valoracion);
	}
```

```
	private static Boolean parseaAdaptado(String adaptadoStr) {
		Boolean res = null;
		adaptadoStr = adaptadoStr.toLowerCase();
		if (adaptadoStr.equals("si")) {
			res = true;
		} else if (adaptadoStr.equals("no")) {
			res = false;
		}
		return res;
	}

	private static Set<Producto> parseaMenu(String menuStr) {
		String [] trozos = menuStr.split(SEPARADOR_MENU);
		Set<Producto> res = new HashSet<>();
		for (String trozo: trozos) {
			res.add(parseaProducto(trozo.trim()));
		}
		return res;
	}

	private static Producto parseaProducto(String productoStr) {
		String [] trozos = productoStr.split(SEPARADOR_PRODUCTO);
		Checkers.check("Formato producto no válido <"+productoStr+">",
				trozos.length == 2);

		String nombre = trozos[0].trim();
		Double precio = Double.parseDouble(trozos[1].trim());
		return new Producto(nombre, precio);
	}

	private static TipoCaseta parseaTipo(String tipoStr) {
		return TipoCaseta.valueOf(tipoStr.toUpperCase());
	}
```

Ejercicio 4: Tratamientos secuenciales (7 ptos)

Tipo Feria

```
	public Integer getNumeroCasetas() {
		return casetas.size();
	}

	public void incorporaCaseta(CasetaFeria cf) {
		casetas.add(cf);
	}

	public void eliminaCaseta(CasetaFeria cf) {
		casetas.remove(cf);
	}
```

Tipo FeriaGestion (herencia -> constructor + toString)

```
public class FeriaGestion extends Feria {

	public FeriaGestion(Set<CasetaFeria> casetas) {
		super(casetas);
	}
```

```
	public String toString() {
		String s = getCasetas().stream()
				.map(Object::toString)
				.collect(Collectors.joining("\n"));
		return super.toString() + "\n" + s;
	}
}
```

Apartado 1 getPrecioMaxProducto – 1.5 Ptos

```
public Map<String, Double> getPrecioMaxProducto(TipoCaseta tipo) {
	Map<String, Double> res = new HashMap<>();
	for (CasetaFeria c: getCasetas()) {
		if (c.getTipo().equals(tipo)) {
			for (Producto p: c.getMenu()) {
				String clave = p.nombre();
				Double valor = p.precio();
				if (!res.containsKey(clave)
						|| valor > res.get(clave)) {
					res.put(clave, valor);
				}
			}
		}
	}
	return res;
}
```

Apartado 2 getValoracionMediaTipoCasetaAccesible – 1 Pto

```
public Double getValoracionMediaTipoCasetaAccesible
		(TipoCaseta tipo, Boolean accesible) {
	return getCasetas().stream()
			.filter(c -> c.getTipo().equals(tipo) &&
			c.getAccesoAdaptado().equals(accesible))
			.mapToDouble(CasetaFeria::getValoracion)
			.average()
			.orElse(0.0);
}

// También
public Double getValoracionMediaTipoCasetaAccesible
		(TipoCaseta tipo, Boolean accesible) {
	return getCasetas().stream()
			.filter(c -> c.getTipo().equals(tipo) &&
				c.getAccesoAdaptado().equals(accesible))
			.collect(Collectors.averagingDouble(
				CasetaFeria::getValoracion));
}
```

Apartado 3 getNombreProductos – 1 Pto

```
public SortedSet<String> getNombresProductos(Double umbral) {
	return getCasetas().stream()
			.flatMap(c -> c.getMenu().stream())
			.filter(p -> p.precio() < umbral)
			.map(Producto::nombre)
			.collect(Collectors.toCollection(TreeSet::new));
}
```

Apartado 4 getTopNCasetasPorMusica – 1,5 Ptos

```
public Map<String, List<String>> getTopNCasetasPorMusica
            (String producto, Integer n) {

      Predicate<CasetaFeria> pred = c -> c.getMenu().stream()
          .anyMatch(pr -> pr.nombre().equals(producto));

      return getCasetas().stream()
                  .filter(pred)
                  .collect(Collectors.groupingBy(
                              CasetaFeria::getMusica,
                              Collectors.collectingAndThen(
                                  Collectors.toList(),
                                  casetas -> getTopNCasetas(casetas, n))
                              ));
}

private List<String> getTopNCasetas(List<CasetaFeria> casetas, Integer n) {

      Comparator<CasetaFeria> cmp =
            Comparator.comparing(CasetaFeria::getValoracion);

      return casetas.stream()
                  .sorted(cmp.reversed())
                  .limit(n)
                  .map(CasetaFeria::getNombre)
                  .collect(Collectors.toList());
}
```

Apartado 5 getCalleMasAforo – 1,5 Ptos

```
public Character getCalleMasAforo () {

      Map<Character, Integer> m = getCasetas().stream()
                  .collect(Collectors.groupingBy(
                        cf -> cf.getModulo().toString().charAt(0),
                        Collectors.summingInt(CasetaFeria::getAforo)));

      return m.entrySet().stream()
                  .max(Map.Entry.comparingByValue())
                  .map(Map.Entry::getKey)
                  .get();
}
```

TESTS

```
public class TestFeria {

  public static void main(String[] args) {
      Set<CasetaFeria> scf = FactoriaFeria.leeCasetas("data/casetas.csv");
      testFactoria(scf);
      FeriaGestion fg = new FeriaGestion(scf);

      System.out.println("Ejercicio 1"+ "=".repeat(60));
      testGetPrecioMaxProducto(fg, TipoCaseta.PUBLICA);
      testGetPrecioMaxProducto(fg, TipoCaseta.PRIVADA);
```

```
        System.out.println("Ejercicio 2"+ "=".repeat(60));
        testGetValoracionMediaTipoCasetaAccesible (fg,
                                                   TipoCaseta.PRIVADA, true);
        testGetValoracionMediaTipoCasetaAccesible (fg,
                                                   TipoCaseta.PUBLICA, true);
        testGetValoracionMediaTipoCasetaAccesible (fg,
                                                    TipoCaseta.PRIVADA, false);
        testGetValoracionMediaTipoCasetaAccesible (fg,
                                                    TipoCaseta.PUBLICA, false);

        System.out.println("Ejercicio 3"+ "=".repeat(60));
        testGetNombresProductos(fg, 5.0);
        testGetNombresProductos(fg, 10.0);

        System.out.println("Ejercicio 4"+ "=".repeat(60));
        testGetTopNCasetasPorMusica (fg, "rebujito", 3);
        testGetTopNCasetasPorMusica (fg, "calamares", 3);

        System.out.println("Ejercicio 5"+ "=".repeat(60));
        testGetCalleMasAforo (fg);
    }

    private static void testGetCalleMasAforo(FeriaGestion fg) {
      try {
          Character res = fg.getCalleMasAforo();
          String msg = String.format("La calle (dígito inicial del número de
                                 módulo) con mayor aforo total es: %s", res);
          System.out.println(msg);
      } catch (Exception e) {
          System.out.println(">>>Capturada excepcion inesperada" +
                                                          e.getMessage());
      }
    }

    private static void testGetTopNCasetasPorMusica(FeriaGestion fg,
                                                    String producto, Integer n) {
      try {
          Map<String, List<String>> res = fg.getTopNCasetasPorMusica(producto,
                                                                  n);
          String msg = String.format("Para el producto %s, las top %d casetas
                               por tipo de música son:\n%s", producto, n, res);
          System.out.println(msg);
      } catch (Exception e) {
          System.out.println(">>>Capturada excepcion inesperada" +
                                                        e.getMessage());
      }
    }

    private static void testGetNombresProductos(FeriaGestion fg,
                                                          Double umbral){
      try {
          SortedSet<String> res = fg.getNombresProductos(umbral);
          String msg = String.format(
                      "Los nombres de los productos cuyo precio no superan en
                       alguna caseta los %.2f euros son:\n %s", umbral, res);
          System.out.println(msg);
      } catch (Exception e) {
          System.out.println(">>>Capturada excepcion inesperada" +
                                                        e.getMessage());
      }
    }
```

```
private static void testGetValoracionMediaTipoCasetaAccesible(
                    FeriaGestion fg, TipoCaseta tipo, Boolean accesible) {
  try {
      Double res = fg.getValoracionMediaTipoCasetaAccesible(tipo,
                                                            accesible);
      String msg = String.format(
                        "La valoración media para las casetas con
           accesibilidad = %s y de tipo %s es: %f", accesible, tipo, res);
      System.out.println(msg);
  } catch (Exception e) {
      System.out.println(">>>Capturada excepcion inesperada" +
                                                          e.getMessage());
  }
}

private static void testGetPrecioMaxProducto(FeriaGestion fg,
                TipoCaseta tipo) {
  try {
      Map<String, Double> res = fg.getPrecioMaxProducto(tipo);
      String msg = String.format("El precio máximo por producto para las
                casetas de tipo %s es\n%s", tipo, res);
      System.out.println(msg);
  } catch (Exception e) {
      System.out.println(">>>Capturada excepcion inesperada" +
                                                        e.getMessage());
  }
}

private static void testFactoria(Set<CasetaFeria> scf) {
  try {
      String msg = String.format("Se han leido %d casetas", scf.size());
      System.out.println(msg);
      mostrarCasetas(scf);
  } catch (Exception e) {
      System.out.println(">>>Capturada excepcion inesperada" +
                                                      e.getMessage());
  }
}

private static void mostrarCasetas(Collection<CasetaFeria> scf) {
  scf.stream().forEach(System.out::println);

}
```